Worcester State University Computer Science - Fall 2026

CS 135 – Introduction to Programming

This is a LASC course and can be taken to satisfy the Quantitative Reasoning requirement.

Credit and Contact Hours

3 credits Lecture: 3 hours/week

Instructor Eldi Musha
Email: emusha@worcester.edu

Office Hours Tuesday: 03:45 pm – 04:15 pm
Thursday: 03:45 pm – 04:15 pm

The office hours will be held in the Computer Science Department area.

If you have any other questions outside of these meeting times and/or need any of my help, please e-mail me your questions and I will help you by e-mail ASAP (Please consider meeting after the lecture).

Catalog Course Description

Introduction to procedural programming. Design, code, test and debug simple procedural programs. No prior programming experience required.

Semester offered: Every Fall and every Spring

Grading

20% Attendance and Lab Work in class
50% Exams (Exam 1 and Exam 2)
30% Final Project

The book used for this class is:

Link to Python book

Overall Course Grades Scale

Percentage score	Letter grade
92-100	A
89-91	A-

86-88	**B+**
82-85	**B**
79-81	**B-**
76-78	**C+**
73-75	**C**
69-72	**C-**
66-68	**D+**
60-65	**D**
55-59	**D-**
0-54	**E**

Where Does This Course Lead?

- Requirement for the CS Major and CS Minor
- Requirement for Bioinformatics Minor
- Elective for Minor in STEM
- CS-140 Object Oriented Programming

Workload Expectations

You should expect to spend, on average, 9 hours per week on this class.

You will spend 3 hours per week in class. In addition, you should expect to spend, on average, at least 6 hours per week outside of class during the semester reading the textbook, studying, completing coursework, and practicing in order to master the concepts covered in the course. (See Definition of the Credit Hour)

Definition of the Credit Hour

Worcester State University follows a modified Carnegie Foundation definition of the credit-hour, known as the "Carnegie Unit." For Worcester State courses, 1 credit hour is equivalent to 1 hour of classroom instruction coupled with a minimum of 2 hours of expected coursework outside of the classroom by the student, over a semester of approximately 15 weeks. Please note that "1 hour of classroom instruction" is actually 50 minutes. A 3-credit course would thus entail approximately 45 classroom hours of instruction (= approximately 37.5 actual hours in class per semester) and a minimum of approximately 90 hours of work outside of class. Courses offered in abbreviated terms, such as summer and winter sessions, are the academic equivalent of courses offered ina full semester format. Students receive one credit for each credit hour of courses taken.

Prerequisites

Familiarity with basic computer operations. Pass math placement test with code of 3 or above, or a passing grade in a college level math course or take MA-099 in the same semester.
If you do not have this background, you should not take this course.

Course-Level Student Learning Outcomes

After successful completion of this course, students will be able to:

- Understand the basic syntax and semantics of a procedural programming language. (Emphasis)
- Model an abstraction as a procedural program. (Emphasis)
- Manipulate integer and floating point values. (Mastery)
- Code, test and debug simple procedural programs given a program design/specification. (Emphasis)
- Code, test and debug simple functions to access and manipulate data. (Emphasis)
- Accept user input from the keyboard, as command-line arguments and read data from a text file. (Emphasis)
- Generate output to the screen and to a text file. (Emphasis)
- Trace, code, test, and debug simple recursive functions and procedures. (Introduced)
- Manipulate and compare strings. (Emphasis)
- Store and access data in lists and dictionaries. (Emphasis)
- Perform simple exception handling. (Introduced)
- Write code representing conditional and repetition control structures. (Mastery)
- Trace code to determine the behavior of the code (Emphasis)
- Draw a reference diagram representing memory and the object/reference relationships (Emphasis)
- Write simple unit tests (Introduced)
- Document code for other programmers, users of code, and users of programs (Emphasis)
- Write programs that can display data in a variety of formats including tables, graphs and charts. (Emphasis)
- Find and evaluate third-party program modules and incorporate them into their own programs. (Introduced)
- Understand and modify code written by others. (Emphasis)

LASC Student Learning Outcomes

This course will contribute to the students' ability to:

- Demonstrate effective oral and written communication.
- Employ quantitative and qualitative reasoning.
- Apply skills in critical thinking.
- Apply skills in information literacy.
- Understand the roles of science and technology in our modern world.
- Understand how scholars in various disciplines approach problems and construct knowledge.

LASC Quantitative Reasoning Objectives

This course will:

1. Acquaint students with formal systems, procedures and sequences of operation.
2. Strengthen students' understanding of variables and functions.
3. Apply mathematical techniques to the analysis and solution of real-life problems.
4. Emphasize the importance of accuracy, including precise language and careful definitions of mathematical concepts.
5. Understand both the underlying principles and practical applications of one or more fields of mathematics.
6. Strengthen understanding of the relationship between algebraic and graphical representations.

Computer Science Minor – Program Learning Outcomes

Graduates of the Computer Science minor will be able to:

- Understand basic concepts in variety of Computer Science fields
- Understand basic syntax of programming languages and apply a programming language to solve real-word problems
- Understand the fundamentals of networks, protocols, the Internet and computer security.
- Analyze a problem, develop/design multiple solutions, and evaluate and document the solutions based on the requirements.
- Communicate effectively both in written and oral form.
- Learn new models, techniques, and technologies as they emerge and appreciate the necessity of such continuing professional development.
- Identify professional and ethical considerations and apply ethical reasoning to technological solutions to problems.

Computer Science Major - Program-Level Student Learning Outcomes

This course addresses the following outcomes of the Computer Science Major: Upon successful completion of the Major in Computer Science, students will be able to:

- Analyze a problem, develop/design multiple solutions and evaluate and document the solutions based on the requirements. (Introductory Level)
- Communicate effectively both in written and oral form. (Introductory Level)
- Demonstrate an understanding of and appreciation for the importance of negotiation, effective work habits, leadership, and good communication with teammates and stakeholders. (Introductory Level)
- Learn new models, techniques, and technologies as they emerge and appreciate the necessity of such continuing professional development. (Introductory Level)

Course Topics

The course outline will be covered on a best-effort basis, subject as always to time limitations as the course progresses.

- How computers represent and process information
- Algorithms: What they are and how to develop one
- Requirements: Writing them, why they are important
- String Operations and Comparisons

- Functions: Parameter passing, Return Values, Anonymous and Lambda Functions
- Sequences: Lists, Tuples and Dictionaries
- Text Processing
- Input/Output: Terminal and File
- Exceptions and Error Handling
- Testing and Debugging
- Translation: Compilation versus Interpretation
- Syntax and Semantics
- Documentation
- Data Types: Variables, Assignment, Operators, Order of Operations, Conversion between Types,
- Control Flow
- Repetition
- Selection: Booleans and Boolean Expressions
- Reading Code for Understanding
- Modules: User Created, System or Standard Modules
- List Processing : Map, Filter and List Comprehension
- Simple recursions

Tentative Schedule: Schedule might change slightly later on as appropriate.

Week		In Class
1	Thursday 9/3 Class 1	Syllabus Introduction to Computers and Programming
2	Tuesday 9/8 Class 2	How Computers Store Data How a program works The Program Development Cycle The Python Environment
2	Thursday 9/10 Class 3	**Input, Processing, and Output** Variables Numeric Data Types and Literals Operators for calculations Data Type Conversions String Concatenations Escape Characters
3	Tuesday 9/15 Class 4	**Control Flow: Branching using ifs** Algorithms If If else Nested ifs: if-elif-else Boolean/logical Operators
3	Thursday 9/17 Class 5	**Lab time**
4	Tuesday 9/22 Class 6	**Loops** While For Infinite Loops
4	Thursday 9/24 Class 7	**Lab Time**
5	Tuesday 9/29 Class 8	**Functions** Void functions Local variables Parameter passing Global Variables The math module Functions and modules organization

5	Thursday 10/1 Class 9	**Lab Time**
6	Tuesday 10/6 Class 10	**Exam 1 Introduction to Programming, I/O, Control Flow and Loops**
6	Thursday 10/8 Class 11	**Lists and Tuples**
7	Tuesday 10/13 Class 12	**Lab Time**
8	Thursday 10/15 Class 13	**File Handling** Input Output Exceptions Loops and Files Records
8	Tuesday 10/20 Class 14	**Lab Time**
9	Thursday 10/22 Class 15	**Strings** Basic String Operations String Slicing Manipulation Strings
9	Tuesday 10/27 Class 16	**Lab Time**
10	Thursday 10/29 Class 17	**Dictionaries and Sets**
10	Tuesday 11/3 Class 18	**Lab Time**
11	Thursday 11/5 Class 19	**Exam 2 Functions, Lists, Tuples, Strings, Dictionaries, Sets and Files**
11	Tuesday 11/10 Class 20	**Classes and OO Programming** Classes Instances Inheritance
12	Thursday 11/12 Class 21	**Project Introduction** Assign Teams
12	Tuesday 11/17 Class 22	**Project Work**
13	Thursday 11/19 Class 23	**Project Work**
14	Tuesday 11/24 Class 24	**Project Work**
14	Tuesday 12/1 Class 25	**Project Presentation Time**
15	Thursday 12/3 Class 26	**Project Presentations**